Importing Libraries

In [65]:
```
import time
import nltk
import os, string
import numpy as np
import pandas as pd
import seaborn as sns
import emoji, re, nltk
from textblob import TextBlob
from wordcloud import WordCloud
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
from sklearn.decomposition import PCA
from sklearn.svm import SVC, LinearSVC
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet, stopwords
from gensim.models import word2vec, FastText
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import train_test_split
from imblearn.under_sampling import RandomUnderSampler
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

# from modelling...
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, roc_auc_score, classification_report
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Bidirectional, Dense, Dropout
from tensorflow.keras.optimizers import Adam
```

Loading the data

- dividing into chunks and loading because the file is too big to load at once.
- concatenate the chunks into a single data-frame

In [4]:
```
file_path = "E:\\Study\\LCIT\\NLP-assignment\\yelp_academic_dataset_review.json"

# to store chunks
review_chunks = []

# Read the JSON file
for chunk in pd.read_json(file_path, lines=True, chunksize=10000):
    review_chunks.append(chunk)

# Concate all chunks into a single DataFrame
review_data = pd.concat(review_chunks, ignore_index=True)
```

Lets start our EDA.

- plotting target variable
- plotting review length in text column
- plotting word cloud to see most common words

In [5]:
```
review_data.shape
```

Out[5]:
```
(6990280, 9)
```

In [6]:
```
review_data.head(10)
```

Out[6]:

	review_id	user_id	business_id	stars	useful	fun
0	KU_O5udG6zpxOg-VcAEodg	mh_-eMZ6K5RLWhZyISBhwA	XQfwVwDr-v0ZS3_CbbE5Xw	3	0	
1	BiTunyQ73aT9WBnpR9DZGw	OyoGAe7OKpv6SyGZT5g77Q	7ATYjTIgM3jUlt4UM3IypQ	5	1	
2	saUsX_uimxRlCVr67Z4Jig	8g_iMtfSiwikVnbP2etR0A	YjUWPpI6HXG530lwP-fb2A	3	0	
3	AqPFMleE6RsU23_auESxiA	_7bHUi9Uuf5__HHc_Q8guQ	kxX2SOes4o-D3ZQBkiMRfA	5	1	
4	Sx8TMOWLNuJBWer-0pcmoA	bcjbaE6dDog4jkNY91ncLQ	e4Vwtrqf-wpJfwesgvdgxQ	4	1	
5	JrIxlS1TzJ-iCu79ul40cQ	eUta8W_HdHMXPzLBBZhL1A	04UD14gamNjLY0IDYVhHJg	1	1	
6	6AxgBCNX_PNTOxmbRSwcKQ	r3zeYsv1XFBRA4dJpL78cw	gmjsEdUsKpj9Xxu6pdjH0g	5	0	
7	_ZeMknuYdlQcUqng_Im3yg	yfFzsLmaWF2d4Sr0UNbBgg	LHSTtnW3YHCeUkRDGyJOyw	5	2	
8	ZKvDG2sBvHVdF5oBNUOpAQ	wSTuiTk-sKNdcFyprzZAjg	B5XSoSG3SfvQGtKEGQ1tSQ	3	1	
9	pUycOfUwM8vqX7KjRRhUEA	59MxRhNVhU9MYndMkz0wtw	gebiRewfieSdtt17PTW6Zg	3	0	

In [7]:
```
review_data.info()
print("----------------------------------------------------")
#  missing values
print(review_data.isnull().sum())
print("----------------------------------------------------")
# Unique values
review_data.nunique()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6990280 entries, 0 to 6990279
Data columns (total 9 columns):
 #   Column       Dtype
---  ------       -----
 0   review_id    object
 1   user_id      object
 2   business_id  object
 3   stars        int64
 4   useful       int64
 5   funny        int64
 6   cool         int64
 7   text         object
 8   date         datetime64[ns]
dtypes: datetime64[ns](1), int64(4), object(4)
memory usage: 480.0+ MB
----------------------------------------------------
review_id      0
user_id        0
business_id    0
stars          0
useful         0
funny          0
cool           0
text           0
date           0
dtype: int64
----------------------------------------------------
```

Out[7]:
```
review_id      6990280
user_id        1987929
business_id     150346
stars                5
useful             259
funny              201
cool               210
text           6974127
date           6885665
dtype: int64
```

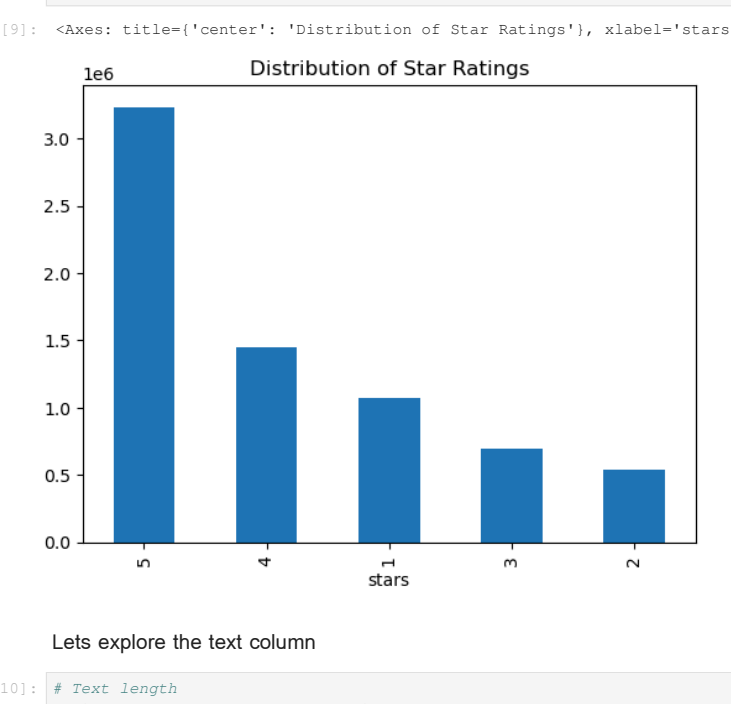
Plotting the target variable will help us understand the data better.

In [8]:
```
# Count the number of reviews for each star rating
star_counts = review_data['stars'].value_counts().sort_index()
print(star_counts)
```

```
stars
1    1069561
2     544240
3     691934
4    1452918
5    3231627
Name: count, dtype: int64
```

In [9]:
```
review_data['stars'].value_counts().plot(kind="bar", title="Distribution of Star Ratings")
```

Out[9]:
```
<Axes: title={'center': 'Distribution of Star Ratings'}, xlabel='stars'>
```

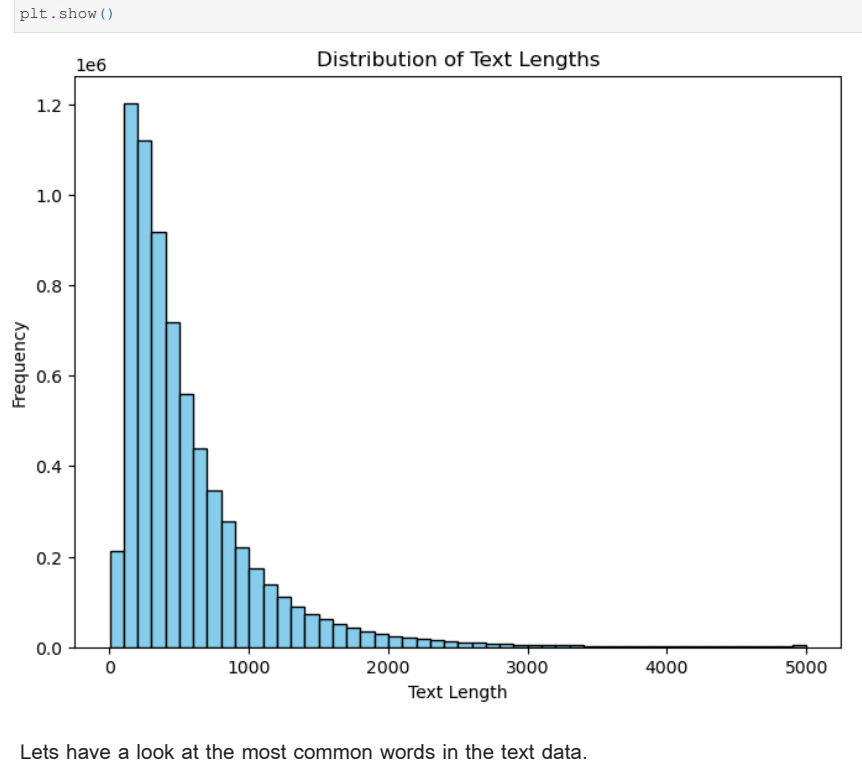


Lets explore the text column

In [10]:
```
# Text length
review_data["text_length"] = review_data['text'].apply(len)
pd.options.display.float_format = '{:,.2f}'.format
print(review_data["text_length"].describe())
```

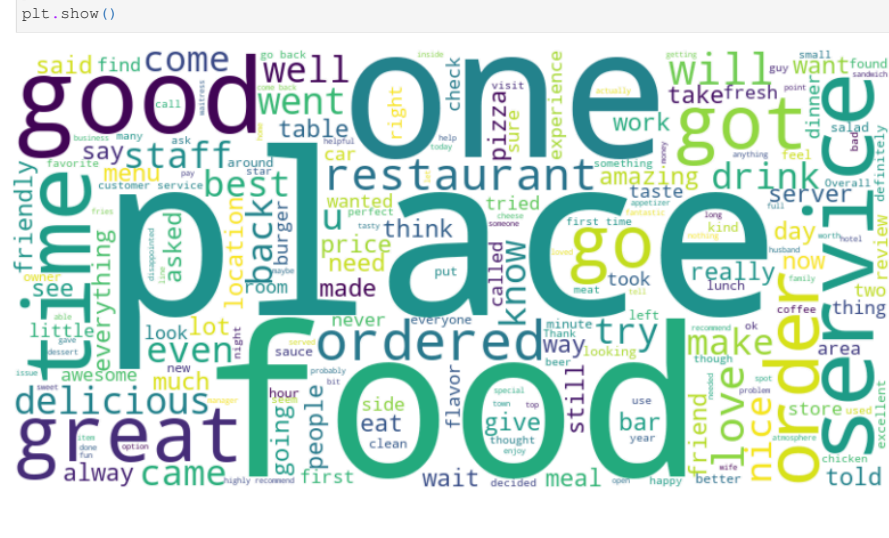
```
count   6,990,280.00
mean          567.76
std           527.26
min             1.00
25%           229.00
50%           406.00
75%           720.00
max         5,000.00
Name: text_length, dtype: float64
```

In [11]:
```
plt.figure(figsize=(8, 6))
plt.hist(review_data["text_length"], bins=50, color='skyblue', edgecolor='black')
plt.title('Distribution of Text Lengths')
plt.xlabel('Text Length')
plt.ylabel('Frequency')
plt.show()
```



Lets have a look at the most common words in the text data.

In [12]:
```
sampled_text = ' '.join(review_data['text'].sample(100000))

wordcloud = WordCloud(width=800, height=400, background_color='white').generate(sampled_tex
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.show()
```



Preprocessing

- Data sampling
- Feature engineering
- Handle Emojis
- Slangs
- Punctuations
- ShortForm
- Spelling Corrections
- Tokenize
- Remove Stop Words
- Negation Handled
- Stemming and Lemmatization

In [36]:
```
def sampled_dataset(df, data_points, balance_data):
    if 'stars' not in df.columns:
        raise ValueError("'stars' column is not in the dataframe")

    if balance_data:
        unique_classes = sorted(df['stars'].unique())

        if 3 in unique_classes:
            # Calculate points for class 3 (double)
            points_for_class_3 = int(data_points * 2 / (len(unique_classes) + 1))
            # Remaining points divided equally among other classes
            points_per_other_class = int((data_points - points_for_class_3) / (len(unique_c

            class_counts = {cls: points_per_other_class for cls in unique_classes if cls !=
            class_counts[3] = points_for_class_3
        else:
            points_per_class = data_points // len(unique_classes)
            class_counts = {cls: points_per_class for cls in unique_classes}

        # Sample data for each class
        balanced_data = []
        for star, count in class_counts.items():
            star_data = df[df['stars'] == star]
            if len(star_data) >= count:  # Check if we have enough samples
                sampled = star_data.sample(n=count, random_state=42)
                balanced_data.append(sampled)
            else:
                # If not enough samples, take all available with replacement
                sampled = star_data.sample(n=count, replace=True, random_state=42)
                balanced_data.append(sampled)

        return pd.concat(balanced_data, ignore_index=True)
    else:
        return df.sample(n=data_points, random_state=42)
```

In [15]:
```
unbalanced_data = sampled_dataset(review_data, 100000, False)
```

In [16]:
```
print("All column names:", review_data.columns)

# Drop all columns except 'text' and 'stars'
review_text_df = review_data[['text', 'stars']]

print(review_text_df.head())
```

```
All column names: Index(['review_id', 'user_id', 'business_id', 'stars', 'useful', 'funny',
       'cool', 'text', 'date', 'text_length'],
      dtype='object')
                                                text  stars
0  If you decide to eat here, just be aware it is...      3
1  I've taken a lot of spin classes over the year...      5
2  Family diner. Had the buffet. Eclectic assortm...      3
3  Wow!  Yummy, different,  delicious.   Our favo...      5
4  Cute interior and owner (?) gave us tour of up...      4
```

In [18]:
```
from slang_dict import abbreviations
def convert_abbrev(word):
    final_word = []
    for i in word.split(" "):
        final_word.append(abbreviations[i.lower()] if i.lower() in abbreviations.keys() els
    return " ".join(final_word)
def convert_abbrev_from_dataset(df, text_column):
    df[text_column] = df[text_column].apply(convert_abbrev)
    return df
```

In [19]:
```
def remove_urls(text):
    url_pattern = re.compile(r'https?://\S+|www\.\S+')
    return url_pattern.sub(r'', text)
def remove_urls_from_dataset(df, text_column):
    df[text_column] = df[text_column].apply(remove_urls)
    return df
```

In [20]:
```
def remove_html(text):
    html_pattern = re.compile('<.*?>')
    return html_pattern.sub(r'', text)
def remove_html_from_dataset(df, text_column):
    df[text_column] = df[text_column].apply(remove_html)
    return df
```

In [21]:
```
def remove_unnecessary_digits(text):
    pattern = r'\b\d+\b|(\d{4}-\d{2}-\d{2})\b\d+\s*(?=\w)'
    # Remove matched patterns
    cleaned_text = re.sub(pattern, '', text)
    # Remove extra whitespace
    cleaned_text = re.sub(r'\s+', ' ', cleaned_text).strip()
    return cleaned_text
def remove_unnecessary_digits_from_dataset(df, text_column):
    df[text_column] = df[text_column].apply(remove_unnecessary_digits)
    return df
```

In [22]:
```
def Negation(sentence):
    '''
    Input: Tokenized sentence (List of words)
    Output: Tokenized sentence with negation handled (List of words)
    '''
    temp = int(0)
    for i in range(len(sentence)):
        if sentence[i-1] in ['not',"n't"]:
            antonyms = []
            for syn in wordnet.synsets(sentence[i]):
                syns = wordnet.synsets(sentence[i])
                w1 = syns[0].name()
                temp = 0
                for l in syn.lemmas():
                    if l.antonyms():
                        antonyms.append(l.antonyms()[0].name())
                max_dissimilarity = 0
                for ant in antonyms:
                    syns = wordnet.synsets(ant)
                    w2 = syns[0].name()
                    syns = wordnet.synsets(sentence[i])
                    w1 = syns[0].name()
                    word1 = wordnet.synset(w1)
                    word2 = wordnet.synset(w2)
                    if isinstance(word1.wup_similarity(word2), float) or isinstance(word1.w
                        temp = 1 - word1.wup_similarity(word2)
                    if temp>max_dissimilarity:
                        max_dissimilarity = temp
                        antonym_max = ant
                        sentence[i] = antonym_max
                        sentence[i-1] = ''
    while '' in sentence:
        try:
            sentence.remove('')
        except:
            pass

    return sentence
```

In [23]:
```
def lemmatize_text_nltk(text):
    lemmatizer = WordNetLemmatizer()
    words = nltk.word_tokenize(text)  # Tokenize the text
    lemmatized_words = [lemmatizer.lemmatize(word) for word in words]
    return ' '.join(lemmatized_words)
def lemmatize_text_nltk_from_dataset(df, text_column):
    df[text_column] = df[text_column].apply(lemmatize_text_nltk)
    return df
```

In [24]:
```
def remove_stopwords(text):
    stop_words = set(stopwords.words('english'))  # Get the set of English stopwords
    words = word_tokenize(text)  # Tokenize the text
    filtered_words = [word for word in words if word.lower() not in stop_words]
    return ' '.join(filtered_words)
def remove_stopwords_from_dataset(df, text_column):
    df[text_column] = df[text_column].apply(remove_stopwords)
    return df
```

In [25]:
```
def cleaning_repeating_char(text):
    return re.sub(r'(.)\1+', r'\1', text)
def cleaning_repeating_char_from_dataset(df, text_column):
    df[text_column] = df[text_column].apply(cleaning_repeating_char)
    return df
```

In [26]:
```
def demojize_text(text):
    return emoji.demojize(text)
def demojize_text_from_dataset(df, text_column):
    df[text_column] = df[text_column].apply(demojize_text)
    return df
```

In [27]:
```
def remove_punctuation(text):
    translator = str.maketrans('', '', string.punctuation)
    return text.translate(translator)
def remove_punctuation_from_dataset(df, text_column):
    df[text_column] = df[text_column].apply(remove_punctuation)
    return df
```

In [28]:
```
def correct_spelling(text):
    textblob = TextBlob(text)
    return str(textblob.correct())
def correct_spelling_from_dataset(df, text_column):
    df[text_column] = df[text_column].apply(correct_spelling)
    return df
```

In [29]:
```
def to_lowercase(text):
    return text.lower()
def lowercase_from_dataset(df, text_column):
    df[text_column] = df[text_column].apply(to_lowercase)
    return df
```

In [41]:
```
def preprocessing(df, text_column):

    balanced_data = demojize_text_from_dataset(df, 'text')
    print("1/12 demojize - DONE!")
    balanced_data = remove_urls_from_dataset(balanced_data, 'text')
    print("2/12 remove_urls - DONE!")
    balanced_data = remove_html_from_dataset(balanced_data, 'text')
    print("3/12 remove_html - DONE!")
    balanced_data = cleaning_repeating_char_from_dataset(balanced_data, 'text')
    print("4/12 cleaning_repeating - DONE!")
    balanced_data = remove_punctuation_from_dataset(balanced_data, 'text')
    print("5/12 remove_punctuation - DONE!")
    balanced_data = remove_unnecessary_digits_from_dataset(balanced_data, 'text')
    print("6/12 remove_unnecessary_digits - DONE!")
    #balanced_data = correct_spelling_from_dataset(balanced_data, 'text')
    #print("7/12 correct_spelling - DONE!")# commented out
    balanced_data = remove_stopwords_from_dataset(balanced_data, 'text')
    print("8/12 remove_stopwords - DONE!")
    balanced_data = cleaning_repeating_char_from_dataset(balanced_data, 'text')
    print("9/12 cleaning_repeating - DONE!")
    balanced_data = lemmatize_text_nltk_from_dataset(balanced_data, 'text')
    print("10/12 tokenize_text - DONE!")
    print("11/12 lemmatize_text - DONE!")
    balanced_data = lowercase_from_dataset(balanced_data, 'text')
    print("12/12 lowercase - DONE!")
    return balanced_data
```

In [46]:
```
# Create sentiment categories based on star ratings
def get_sentiment_category(stars):
    if stars >= 4:
        return 'Positive'
    elif stars <= 2:
        return 'Negative'
    else:
        return 'Neutral'
```

In [85]:
```
# here pass the dataset(review_text_df) and
# pick how many data points you want to sample (10000).
# True  -  will balance the data points  with respect to the classes
# False  -  will randomly sample the data points
balanced_data = sampled_dataset(review_text_df, 200000, True)
#balanced_data = sampled_dataset(review_text_df, 100000, True)      # commented out for 2nd

# unbalanced_data = sampled_dataset(review_text_df, 100000, False)   # example for unbalanc

# Display the results
print(balanced_data['stars'].value_counts())
print(unbalanced_data['stars'].value_counts())
```

```
stars
3    66666
1    33333
2    33333
4    33333
5    33333
Name: count, dtype: int64
stars
5    46202
4    20835
1    15235
3     9882
2     7846
Name: count, dtype: int64
```

In [86]:
```
preprocessed_df = preprocessing(balanced_data, 'text')
```

```
1/12 demojize - DONE!
2/12 remove_urls - DONE!
3/12 remove_html - DONE!
4/12 cleaning_repeating - DONE!
5/12 remove_punctuation - DONE!
6/12 remove_unnecessary_digits - DONE!
8/12 remove_stopwords - DONE!
9/12 cleaning_repeating - DONE!
10/12 tokenize_text - DONE!
11/12 lemmatize_text - DONE!
12/12 lowercase - DONE!
```

In [87]:
```
print(preprocessed_df.head())
```

```
                                                text  stars
0  leave dor lady changing rom proped open weird ...      1
1  place joke say al eat fre ber sake amount time...      1
2  bad breakfast grits god puled pork salty margi...      1
3  visited mud house heard excelent cofe great at...      1
4  went thought opened dor locked went leave timo...      1
```

In [88]:
```
# Add sentiment column
preprocessed_df['sentiment'] = preprocessed_df['stars'].apply(get_sentiment_category)

# Display the distribution of sentiments
print("Sentiment Distribution:")
print(preprocessed_df['sentiment'].value_counts())
print("\nPercentage Distribution:")
print(preprocessed_df['sentiment'].value_counts(normalize=True).multiply(100).round(2), "%"
```

```
Sentiment Distribution:
sentiment
Negative    66666
Positive    66666
Neutral     66666
Name: count, dtype: int64

Percentage Distribution:
sentiment
Negative    33.33
Positive    33.33
Neutral     33.33
Name: proportion, dtype: float64 %
```

Data Splitting

In [89]:
```
X = preprocessed_df['text']
y = preprocessed_df['sentiment']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

In [90]:
```
# Label encoding
label_encoder = LabelEncoder()

y_train = label_encoder.fit_transform(y_train)
y_test = label_encoder.transform(y_test)
```

In [ ]:
```
#Download the data

#columns = ['text', 'sentiment']
#df = pd.DataFrame(preprocessed_df, columns=columns)
#df.to_csv('your_dataset.csv', index=False)
#print("Dataset has been saved as 'your_dataset.csv'")
```

```
Dataset has been saved as 'your_dataset.csv'
```

In [91]:
```
max_words = 10000
max_len = 100

tokenizer = Tokenizer(num_words=max_words)
tokenizer.fit_on_texts(X_train)

X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)

# Pad sequences to ensure uniform input size
X_train_pad = pad_sequences(X_train_seq, maxlen=max_len)
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len)

print('Text data has been tokenized and padded.')
```

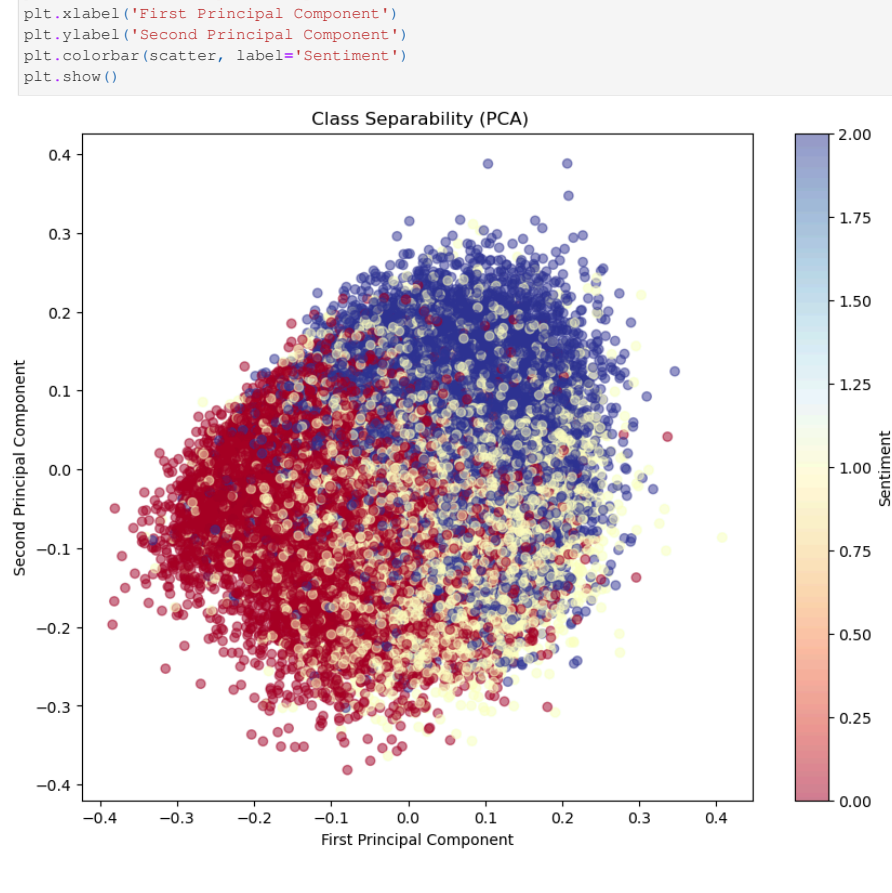
```
Text data has been tokenized and padded.
```

In [92]:
```
print("Training data shape:", X_train_pad.shape)
print("Testing data shape:", X_test_pad.shape)
print("Vocabulary size:", min(max_words, len(tokenizer.word_index) + 1))
```

```
Training data shape: (159998, 100)
Testing data shape: (40000, 100)
Vocabulary size: 10000
```

Checking for class separability

In [77]:
```
tfidf = TfidfVectorizer(max_features=1000)
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_tfidf.toarray())

plt.figure(figsize=(10, 8))
scatter = plt.scatter(X_train_pca[:, 0], X_train_pca[:, 1], c=y_train, cmap='RdYlBu', alpha
plt.title(' Class Separability (PCA)')
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.colorbar(scatter, label='Sentiment')
plt.show()
```



Modelling...

In [78]:
```
# Model parameters
vocab_size = min(max_words, len(tokenizer.word_index) + 1)
embedding_dim = 128
lstm_units = 64

# Build the model
model = Sequential([
    # Embedding Layer
    Embedding(input_dim=vocab_size,
            output_dim=embedding_dim,
            input_length=max_len),

    # Bidirectional LSTM Layer
    Bidirectional(LSTM(lstm_units,
                    return_sequences=True,
                    recurrent_dropout=0.2)),

    # Additional Bidirectional LSTM Layer
    Bidirectional(LSTM(lstm_units,
                    return_sequences=False,
                    recurrent_dropout=0.2)),

    # Dropout for regularization
    Dropout(0.5),

    # Dense hidden layer
    Dense(64, activation='relu'),
    Dropout(0.3),

    # Output layer
    Dense(len(label_encoder.classes_), activation='softmax')
])
```

```
c:\Users\ASUS\anaconda3\Lib\site-packages\keras\src\layers\core\embedding.py:90: UserWarnin
g: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
```

In [79]:
```
# Compile the model
model.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
```

In [80]:
```
start_time = time.time()
history = model.fit(
    X_train_pad,
    y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
training_time = time.time() - start_time
print(f"\nTraining Time: {training_time:.2f} seconds")
```

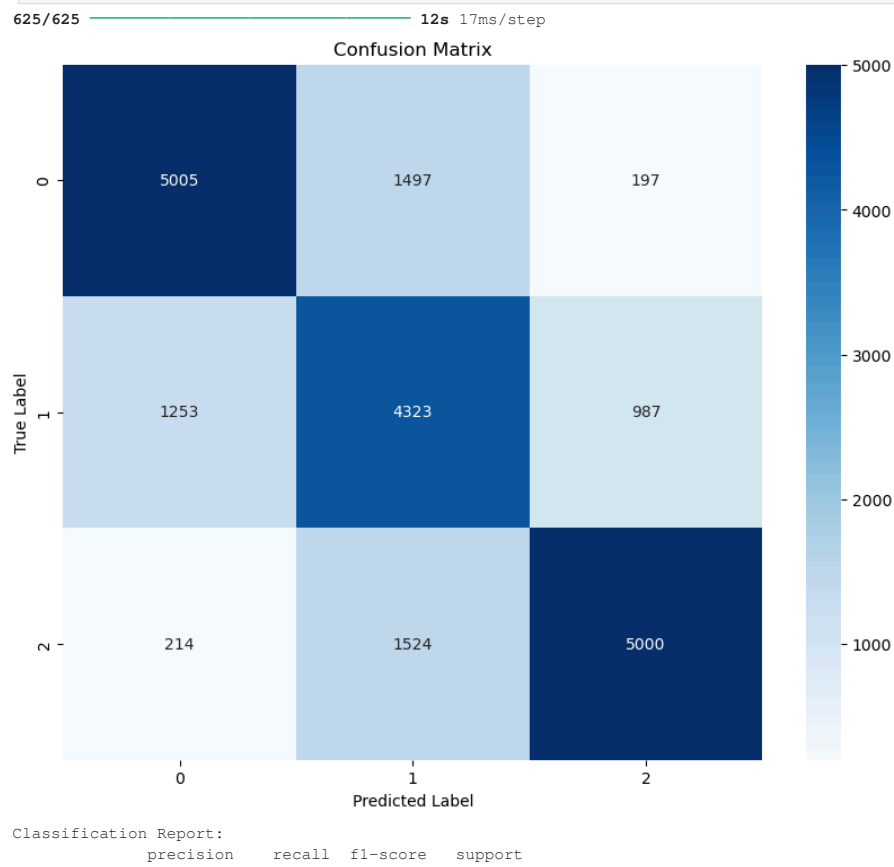
```
Epoch 1/5
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 159s 75ms/step - accuracy: 0.6053 - loss: 0.822
9 - val_accuracy: 0.7194 - val_loss: 0.6431
Epoch 2/5
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 190s 95ms/step - accuracy: 0.7431 - loss: 0.603
3 - val_accuracy: 0.7259 - val_loss: 0.6174
Epoch 3/5
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 182s 91ms/step - accuracy: 0.7758 - loss: 0.534
1 - val_accuracy: 0.7242 - val_loss: 0.6295
Epoch 4/5
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 174s 87ms/step - accuracy: 0.8088 - loss: 0.470
3 - val_accuracy: 0.7236 - val_loss: 0.6731
Epoch 5/5
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 171s 86ms/step - accuracy: 0.8313 - loss: 0.411
8 - val_accuracy: 0.7140 - val_loss: 0.7041

Training Time: 876.38 seconds
```

In [81]:
```
# Print model summary
model.summary()
```

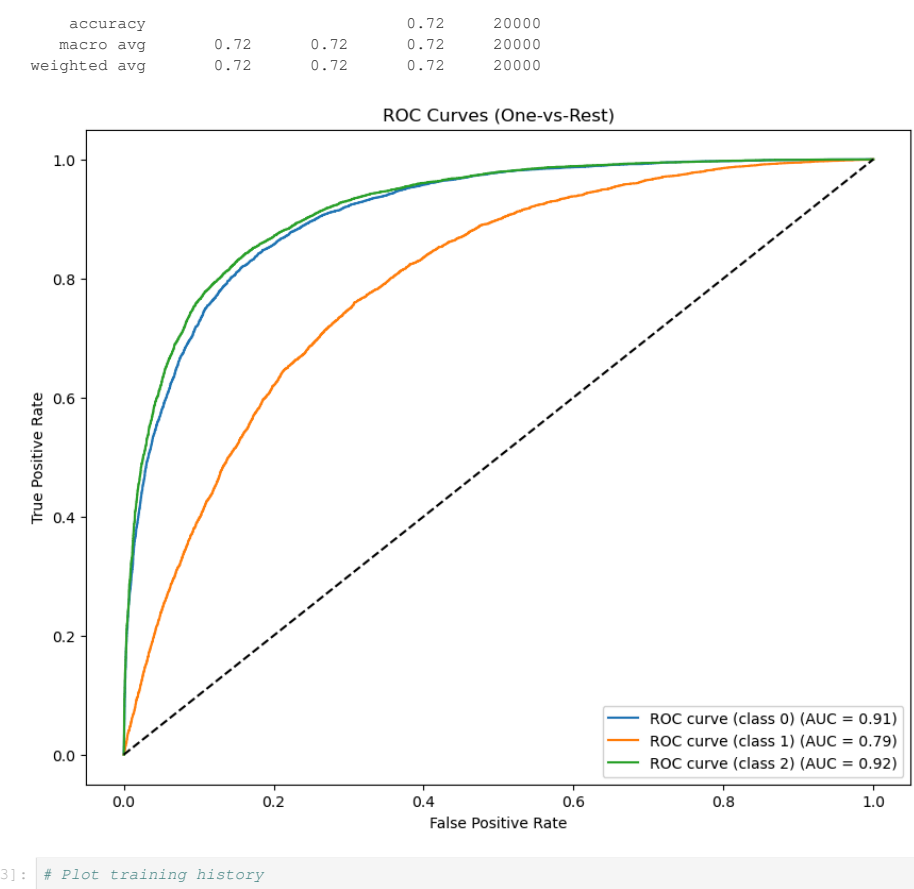
```
Model: "sequential_1"
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ embedding_1 (Embedding)         │ (None, 100, 128)       │     1,280,000 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ bidirectional_2 (Bidirectional) │ (None, 100, 128)       │        98,816 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ bidirectional_3 (Bidirectional) │ (None, 128)            │        98,816 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dropout_2 (Dropout)             │ (None, 128)            │             0 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_2 (Dense)                 │ (None, 64)             │         8,256 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dropout_3 (Dropout)             │ (None, 64)             │             0 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_3 (Dense)                 │ (None, 3)              │           195 │
└─────────────────────────────────┴────────────────────────┴───────────────┘
 Total params: 4,458,251 (17.01 MB)
 Trainable params: 1,486,083 (5.67 MB)
 Non-trainable params: 0 (0.00 B)
 Optimizer params: 2,972,168 (11.34 MB)
```

In [82]:
```
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc

# Get predictions
y_pred = model.predict(X_test_pad)
y_pred_classes = np.argmax(y_pred, axis=1)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_classes)
plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_classes))

# ROC Curve and AUC Score (One-vs-Rest)
n_classes = len(label_encoder.classes_)
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve((y_test == i).astype(int), y_pred[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Plot ROC curves
plt.figure(figsize=(10, 8))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i],
             label=f'ROC curve (class {i}) (AUC = {roc_auc[i]:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves (One-vs-Rest)')
plt.legend(loc="lower right")
plt.show()
```

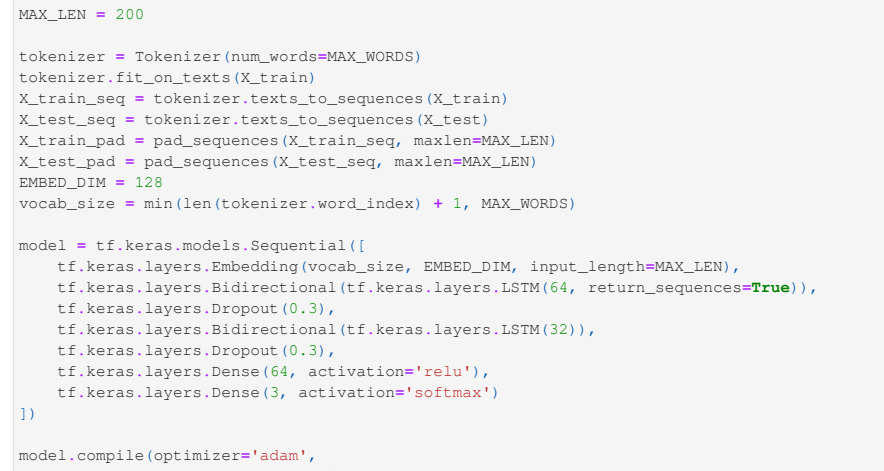
```
625/625 ━━━━━━━━━━━━━━━━━━━━ 12s 17ms/step
```



```
Classification Report:
              precision    recall  f1-score   support

           0       0.77      0.75      0.76      6699
           1       0.59      0.66      0.62      6563
           2       0.81      0.74      0.77      6738

    accuracy                           0.72     20000
   macro avg       0.72      0.72      0.72     20000
weighted avg       0.72      0.72      0.72     20000
```



In [83]:
```
# Plot training history
plt.figure(figsize=(12, 4))

# Plot training & validation accuracy values
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

# Plot training & validation loss values
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.tight_layout()
plt.show()
```



tuning approaches

- we can see over fitting in the model lets regularize using L2 to make the model better, also increasing dropout for last dense layer from 0.3 to 0.5
- I can see in the confusion matrix that the model is getting confused with the adjacent class, I will provide more data so that model can understand the difference better

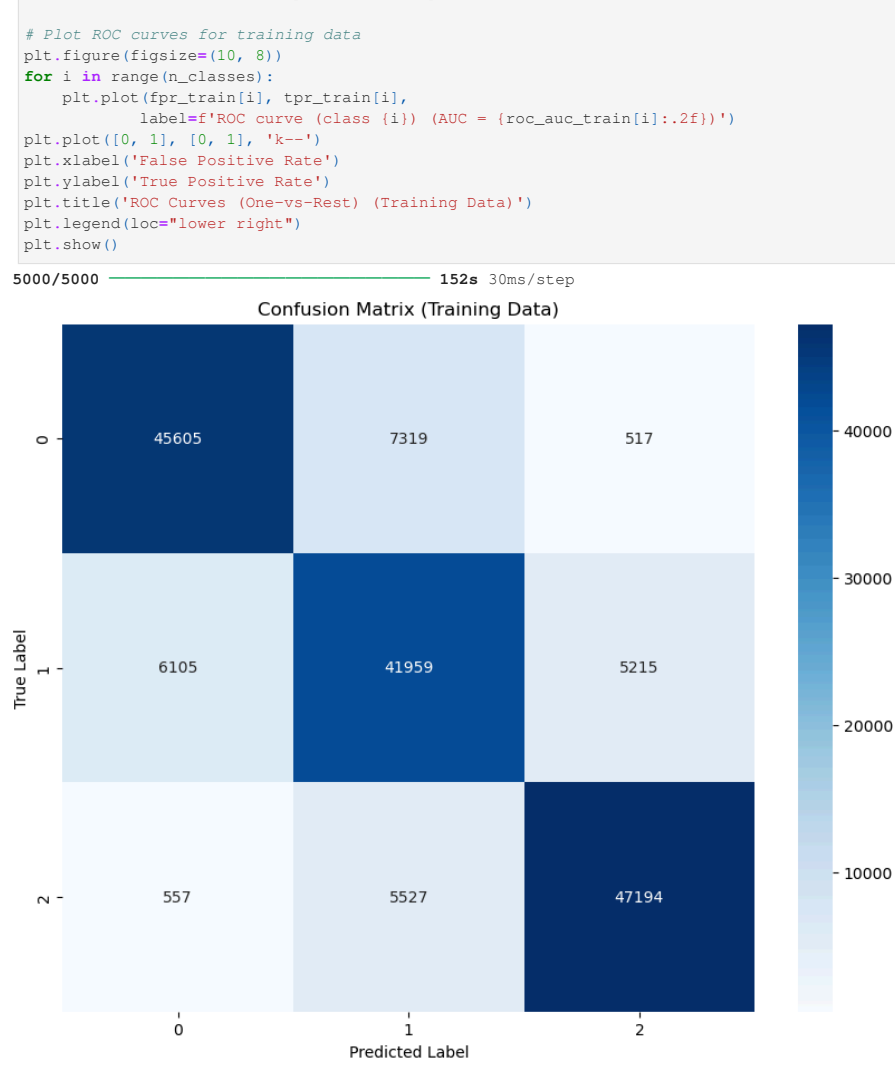
In [95]:
```
import tensorflow as tf

MAX_WORDS = 10000
MAX_LEN = 200

tokenizer = Tokenizer(num_words=MAX_WORDS)
tokenizer.fit_on_texts(X_train)
X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)
X_train_pad = pad_sequences(X_train_seq, maxlen=MAX_LEN)
X_test_pad = pad_sequences(X_test_seq, maxlen=MAX_LEN)
EMBED_DIM = 128
vocab_size = min(len(tokenizer.word_index) + 1, MAX_WORDS)

model = tf.keras.models.Sequential([
    tf.keras.layers.Embedding(vocab_size, EMBED_DIM, input_length=MAX_LEN),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(64, return_sequences=True)),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(32)),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])

model.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
```

In [96]:
```
# Train model
start_time = time.time()
history = model.fit(X_train_pad, y_train, epochs=5, batch_size=64, validation_split=0.2, ve

# Calculate training time
training_time = time.time() - start_time
print(f"\nTraining Time: {training_time:.2f} seconds")
```

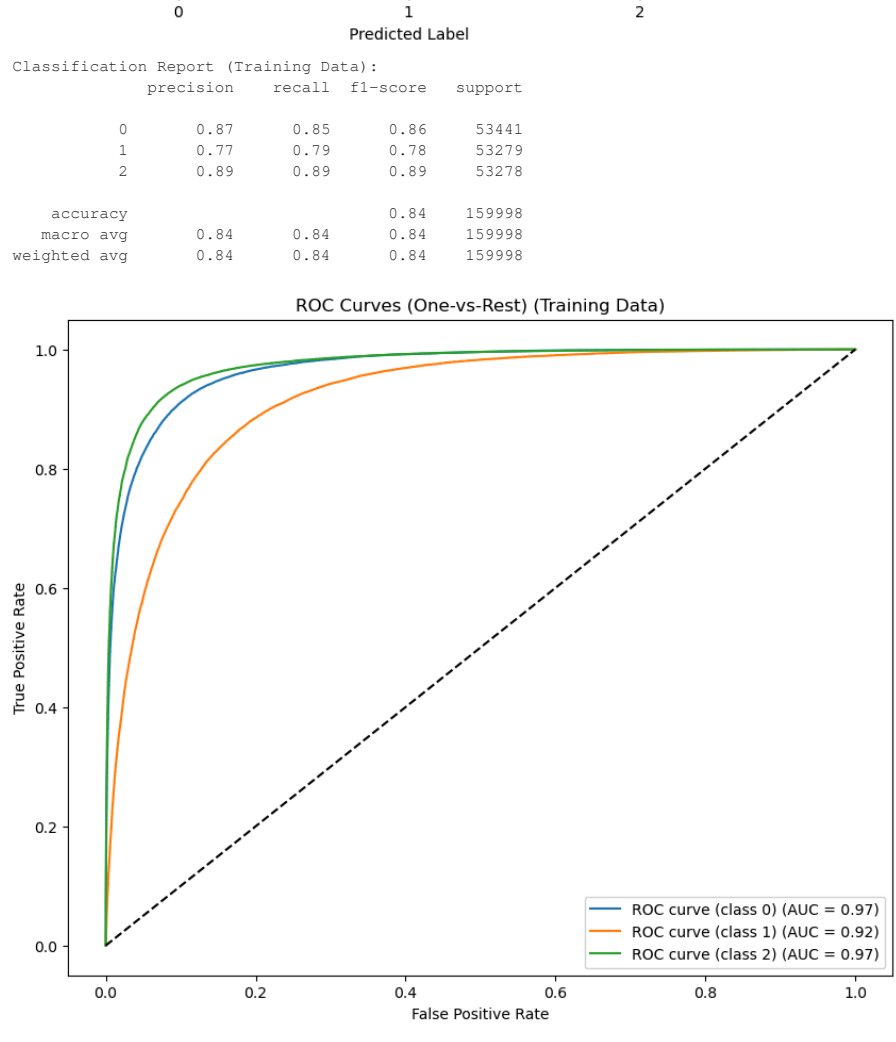
```
Epoch 1/5
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 449s 220ms/step - accuracy: 0.6572 - loss: 0.73
14 - val_accuracy: 0.7344 - val_loss: 0.6010
Epoch 2/5
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 412s 206ms/step - accuracy: 0.7586 - loss: 0.55
55 - val_accuracy: 0.7389 - val_loss: 0.5886
Epoch 3/5
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 432s 216ms/step - accuracy: 0.7872 - loss: 0.49
62 - val_accuracy: 0.7341 - val_loss: 0.6131
Epoch 4/5
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 454s 227ms/step - accuracy: 0.8121 - loss: 0.44
21 - val_accuracy: 0.7308 - val_loss: 0.6293
Epoch 5/5
2000/2000 ━━━━━━━━━━━━━━━━━━━━ 446s 223ms/step - accuracy: 0.8362 - loss: 0.39
17 - val_accuracy: 0.7292 - val_loss: 0.6727

Training Time: 2192.73 seconds
```

In [103…
```
# Get predictions for training data
y_train_pred = model.predict(X_train_pad)
y_train_pred_classes = np.argmax(y_train_pred, axis=1)

# Confusion Matrix for training data
cm_train = confusion_matrix(y_train, y_train_pred_classes)
plt.figure(figsize=(10,8))
sns.heatmap(cm_train, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix (Training Data)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

# Classification Report for training data
print("\nClassification Report (Training Data):")
print(classification_report(y_train, y_train_pred_classes))

# ROC Curve and AUC Score (One-vs-Rest) for training data
n_classes = len(label_encoder.classes_)
fpr_train = dict()
tpr_train = dict()
roc_auc_train = dict()

for i in range(n_classes):
    fpr_train[i], tpr_train[i], _ = roc_curve((y_train == i).astype(int), y_train_pred[:, i
    roc_auc_train[i] = auc(fpr_train[i], tpr_train[i])

# Plot ROC curves for training data
plt.figure(figsize=(10, 8))
for i in range(n_classes):
    plt.plot(fpr_train[i], tpr_train[i],
             label=f'ROC curve (class {i}) (AUC = {roc_auc_train[i]:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves (One-vs-Rest) (Training Data)')
plt.legend(loc="lower right")
plt.show()
```

```
5000/5000 ━━━━━━━━━━━━━━━━━━━━ 152s 30ms/step
```

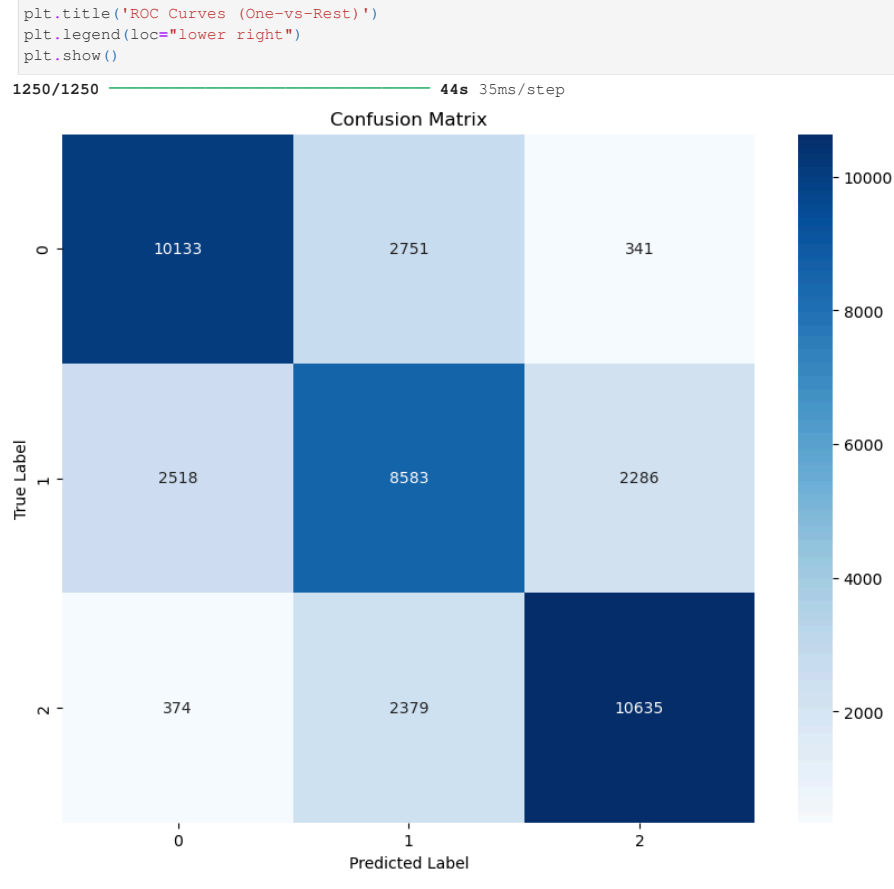


```
Classification Report (Training Data):
              precision    recall  f1-score   support

           0       0.87      0.85      0.86     53441
           1       0.77      0.79      0.78     53279
           2       0.89      0.89      0.89     53278

    accuracy                           0.84    159998
   macro avg       0.84      0.84      0.84    159998
weighted avg       0.84      0.84      0.84    159998
```

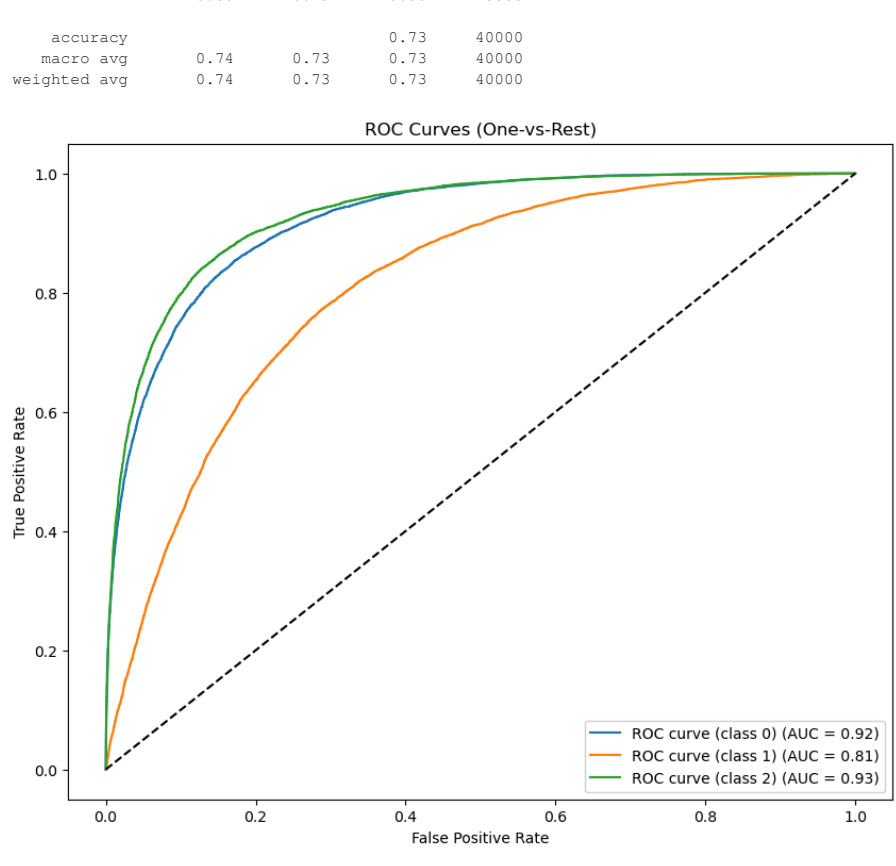


In [97]:
```
# Get predictions
y_pred = model.predict(X_test_pad)
y_pred_classes = np.argmax(y_pred, axis=1)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_classes)
plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_classes))

# ROC Curve and AUC Score (One-vs-Rest)
n_classes = len(label_encoder.classes_)
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve((y_test == i).astype(int), y_pred[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Plot ROC curves
plt.figure(figsize=(10, 8))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i],
             label=f'ROC curve (class {i}) (AUC = {roc_auc[i]:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves (One-vs-Rest)')
plt.legend(loc="lower right")
plt.show()
```

```
1250/1250 ━━━━━━━━━━━━━━━━━━━━ 44s 35ms/step
```

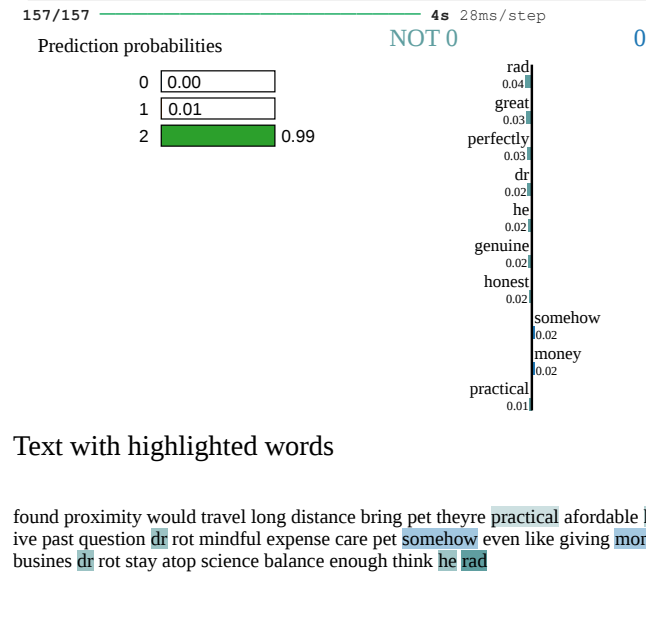


```
Classification Report:
              precision    recall  f1-score   support

           0       0.78      0.77      0.77     13225
           1       0.63      0.64      0.63     13387
           2       0.80      0.79      0.80     13388

    accuracy                           0.73     40000
   macro avg       0.74      0.73      0.73     40000
weighted avg       0.74      0.73      0.73     40000
```



In [102…
```
# Converting tokenized sequences back to text (LIME works better with raw text for text cla
def sequence_to_text(tokenizer, sequence):
    reverse_word_index = {index: word for word, index in tokenizer.word_index.items()}
    return ' '.join([reverse_word_index.get(i, '') for i in sequence])

# Creating a function that wraps the model predictions to work with LIME
def predict_text(texts):
    sequences = tokenizer.texts_to_sequences(texts)
    sequences = pad_sequences(sequences, maxlen=MAX_LEN)
    return model.predict(sequences)

num_classes = len(np.unique(y_train))  # Get the number of unique classes in y_train
explainer = LimeTextExplainer(class_names=[str(i) for i in range(num_classes)])
# Choosing the first index for explanation
idx = 1
raw_text = sequence_to_text(tokenizer, X_test_pad[idx])
# Explain the instance prediction
exp = explainer.explain_instance(raw_text, predict_text, num_features=10, labels=[0])  # Yo
# Visualize the explanation
exp.show_in_notebook(text=True)
```

```
157/157 ━━━━━━━━━━━━━━━━━━━━ 4s 28ms/step
```



Text with highlighted words

theyre great found proximity would travel long distance bring pet theyre practical afordable honest genuine compared vet ive past question dr rot mindful expense care pet somehow even like giving money perfectly content suport busines dr rot stay atop science balance enough think he rad